Online Fashion Retail Store Database System

Emmanuel Madueke · Owner, author and maintainer
Student number: 301912318 · September cohort 2026 · Updated 22 October 2026

GitHub: github.com/EmmanuelMadueke/advanced-database-portfolio

Supporting screenshots: nine-image evidence gallery (saved outputs and diagrams; captured 22 October).

Organisation, users and operations

The case study is a simulated online retailer selling clothing, footwear and accessories. Customers browse variants, manage carts, place orders and track purchases. Administrators maintain products, stock and fulfilment; management uses sales and inventory reports. Major data includes customers, addresses, products, categories, brands, sizes, colours, variants, inventory, carts, orders, items, payment attempts and deliveries.

One scenario connects all five activities. PostgreSQL owns transactional prices, stock and purchase records. MongoDB provides flexible descriptive product enrichment. Data is synthetic; payment outcomes are simulated. No live storefront, payment gateway or cloud deployment is claimed.

1 · Database design and modelling

The relational design contains 16 tables with primary/foreign keys, uniqueness, required values and validation rules. A table-style ERD shows attributes, keys, cardinalities and optionality. Normalisation explains 1NF–3NF and deliberate purchase-time snapshots: later catalogue or address changes must not rewrite an earlier purchase.

PostgreSQL 18.1 implements the schema, integrity triggers, controlled write routines and a restricted demonstration writer role. Checks cover invalid references, duplicate variants, negative stock, invalid prices, lifecycle transitions and checkout consistency. The sample has 12 customers, eight products, 18 variants and 12 orders; recognised sales reconcile to NGN 268,000.

Evidence: ERD image · schema, normalisation and outputs. Fresh checks: 35 integrity/workflow, 10 sample and four scale assertions.

2 · Query processing and optimisation

Twelve reports cover catalogue filtering, stock, customer history, fulfilment, revenue and sales trends. Revenue uses historical purchase amounts and eligible order states; failed payment retries do not duplicate revenue. Thirty result checks cover totals, empty groups, date endpoints and timezone behaviour.

Five reports were measured on a synthetic 10,012-order dataset before indexing, after three reporting indexes, and after a targeted history rewrite. Each phase used a warm-up and five measured plans. Complete ordered-result fingerprints matched. The rewrite aggregates payment attempts for selected customer orders instead of scanning every attempt.

Query results and limits

Fresh run: 17 October	Before median	Final median
Customer history	1.444 ms	0.116 ms
Revenue summary	1.055 ms	0.191 ms

These local measurements support the selected workload changes, not a production speed guarantee. Warm caches, synthetic distribution and instrumentation limit generalisation. Unchanged catalogue/report plans did not justify claiming an index benefit from timing variation alone. Index maintenance adds storage and write costs.

Evidence: optimisation analysis · fresh comparison output.

3 · Transactions and concurrency

A controlled checkout transaction combines order creation, item snapshots, simulated successful payment and inventory deduction. A NGN 49,000 purchase passed eight persistence checks after COMMIT. An injected pre-commit failure followed by ROLLBACK passed six checks for no purchase rows and restored stock. Two persistent sessions passed seven READ COMMITTED visibility checks.

An isolated unsafe last-unit model let two buyers read stock one and commit two purchases, demonstrating why nonnegative stock alone does not prevent overselling. The protected checkout made the second buyer wait on the first, then rejected the depleted-stock purchase. Eleven checks confirmed one successful buyer, stock conservation and unchanged state on replay of known-order finalisation. That replay does not make new-order requests or external charges idempotent.

```
Selected fresh output — protected-sessions.txt:
PASS: loser has no retained order
PASS: stock conserved and unrelated jacket unchanged
11 protected-race checks passed.
```

Evidence: transaction schedules, SQL and outputs.

4 · NoSQL and advanced data models

MongoDB Community 8.0.34 stores three product-enrichment profiles with embedded category-specific materials, care and descriptive features. String IDs preserve SQL product identity. Strict validation checks document shape; importer/reconciliation checks enforce source matching and rules such as material percentages. PostgreSQL retains authority over stock, price, orders and payments.

The fresh import passed 22 checks plus four checks through a new connection. Five finds and a tag aggregation passed 16 checks. A compound multikey index was tested: a hinted diagnostic examined one document and preserved results, but the automatic planner retained the identity scan on this tiny collection. No speedup is claimed. The SQL-to-MongoDB workflow reconciles one exported snapshot; live synchronisation is not implemented.

Evidence: model, implementation and queries · SQL/NoSQL suitability and limits.

5 · Distributed and cloud databases

The permitted discussion path proposes one region with three independent zones. PostgreSQL has one writer, a required synchronous standby and an asynchronous reporting standby. Recent-order reads use the current writer; lagging reports cannot authorise a purchase. Safe failover requires exclusive authority, fencing of the former writer and validated recovery. Loss of the required standby can stall checkout commits.

A three-member MongoDB replica set is proposed for enrichment. Database replication would not refresh copied SQL facts. Sharding is deferred: there is no production capacity evidence requiring it, and a basket spanning inventory shards would need coordinated transactions or an explicit reservation/recovery design. Eight scenarios cover failures, stale reads, ambiguous commits and recovery. All topology and failure behaviour remain proposed and untested.

Evidence: architecture diagram and scenarios · decisions and references.

Reproduction and ownership

A fresh run at commit **5ba7d37** completed all 52 recorded commands on 17 October 2026. Separate logs, plans, versions, manifests and hashes preserve the results; both disposable database runtimes were stopped. Portfolio 5 received documentation checks only. Evidence was committed progressively; the fresh results were published in commit **20387ce**. These are tool-run checks, not independent student-verification claims.

Individual technical reflection

I directed changes to the ERD presentation by supplying a table-style reference, reviewing the revised appearance and requesting explicit explanations of relationship ratios. Presenting keys and attributes alongside one-to-one and one-to-many relationships makes the model easier to inspect: the drawing needs to communicate business rules as well as table names. I also requested an image without the rendering script, and kept development staged through checklist updates and progressive commits.

The technical evidence illustrates why a plausible result needs a stronger check. Nonnegative stock did not prevent two purchases in the unsafe race; the protected test also checked purchase counts and stock conservation. Similarly, query timings were interpreted alongside unchanged results and execution plans. These are useful principles for future database work: specify the business invariant, test failure cases, and retain evidence for each claimed improvement. My recorded review covers the ERD presentation. The runtime tests were executed through Codex; independently reproducing and explaining them remains an area for further personal verification.

AI use declaration and references

OpenAI Codex assisted with requirements, code and documentation drafting, test execution, result analysis, the distributed-design discussion and this summary. I supplied the ERD reference, reviewed its appearance, requested relationship explanations and directed progressive development. Saved checks document tool-executed verification; I do not claim independent execution of those tests. I remain responsible for the accuracy and understanding of the submitted work.

Assessment basis: *MIT 8103 Advanced Database Systems continuous assessment, September cohort 2026*, sections 2–12. Technical sources: PostgreSQL 18 documentation (constraints, EXPLAIN, isolation, replication and recovery); MongoDB Manual (validation, multikey indexes, replica sets and sharding). Activity documents contain claim-level links. Reflection structure draws on the University of Edinburgh's Reflection Toolkit and Monash University's reflective-practice guidance (accessed 17 October 2026). SQL and enrichment datasets are synthetic. PostgreSQL, MongoDB, Docker, Python and Node.js/mongosh support the recorded implementation and checks.